



Software Relevant Tools, Standards, and Code Versioning using GitHub

Name:	CELAYA, Alyzee Yvon N.	Date:	August 22, 2026
Program:	CPE	Section:	B3

A. Learning Outcomes:

At the end of this laboratory, students should be able to:

- 1. Identify software development tools and standards used in software projects.
- 2. Create and manage a GitHub repository.
- 3. Apply Git version control commands.
- 4. Collaborate using GitHub features.
- 5. Demonstrate proper documentation and coding standards.

B. Laboratory Scenario:

You have been hired as a junior software developer in a software company. Your team uses GitHub for source code management and follows coding standards to maintain software quality. Your task is to create a project repository, manage versions of source code, and document your work according to industry standards.

C. Practical Tasks:

1. Software Development Tools and Standards

Create a document named Tools_and_Standards.md containing:

- a. List at least 5 development tools.
- b. Describe the purpose of each tool.
- c. Identify at least three coding standards or best practices used in software development.

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

Link to GitHub Repository File:

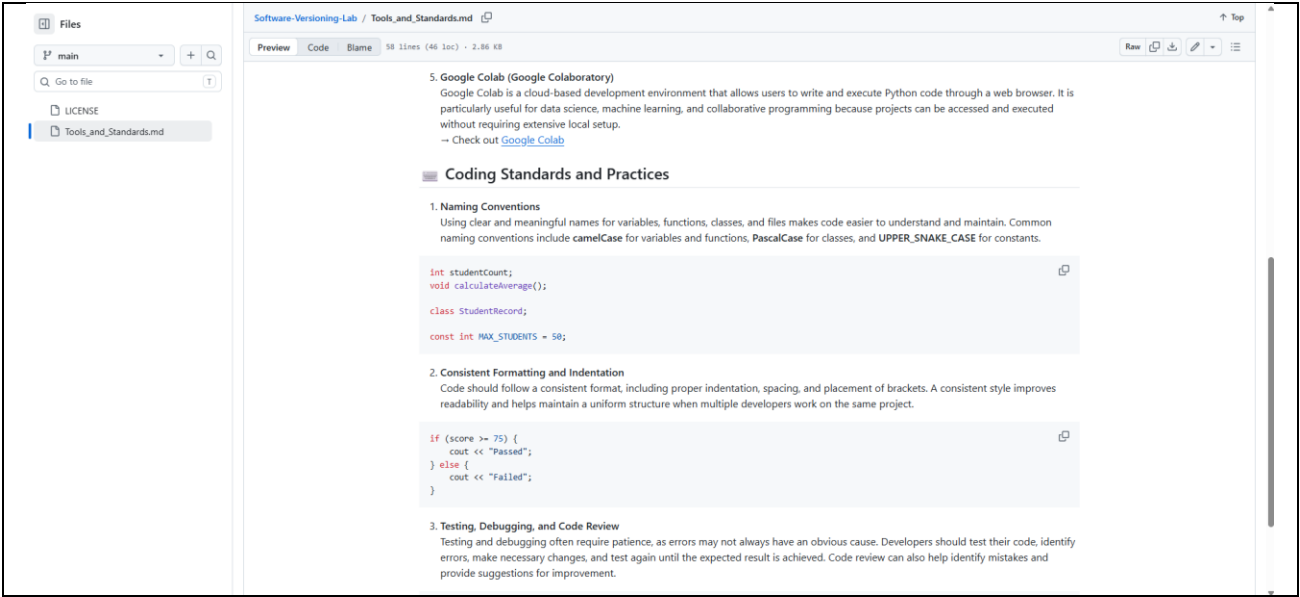
[Tools and Standard.md](#)

Output Preview Code of Tools_and_Standards.md



Laboratory Report 1

Software Relevant Tools, Standards, and Code Versioning using GitHub



2. GitHub Repository Creation
- a. Create a GitHub account (if none exists).

b. Create a public repository named: Software-Versioning-Lab

c. Add the following:

i. README.md

ii. LICENSE

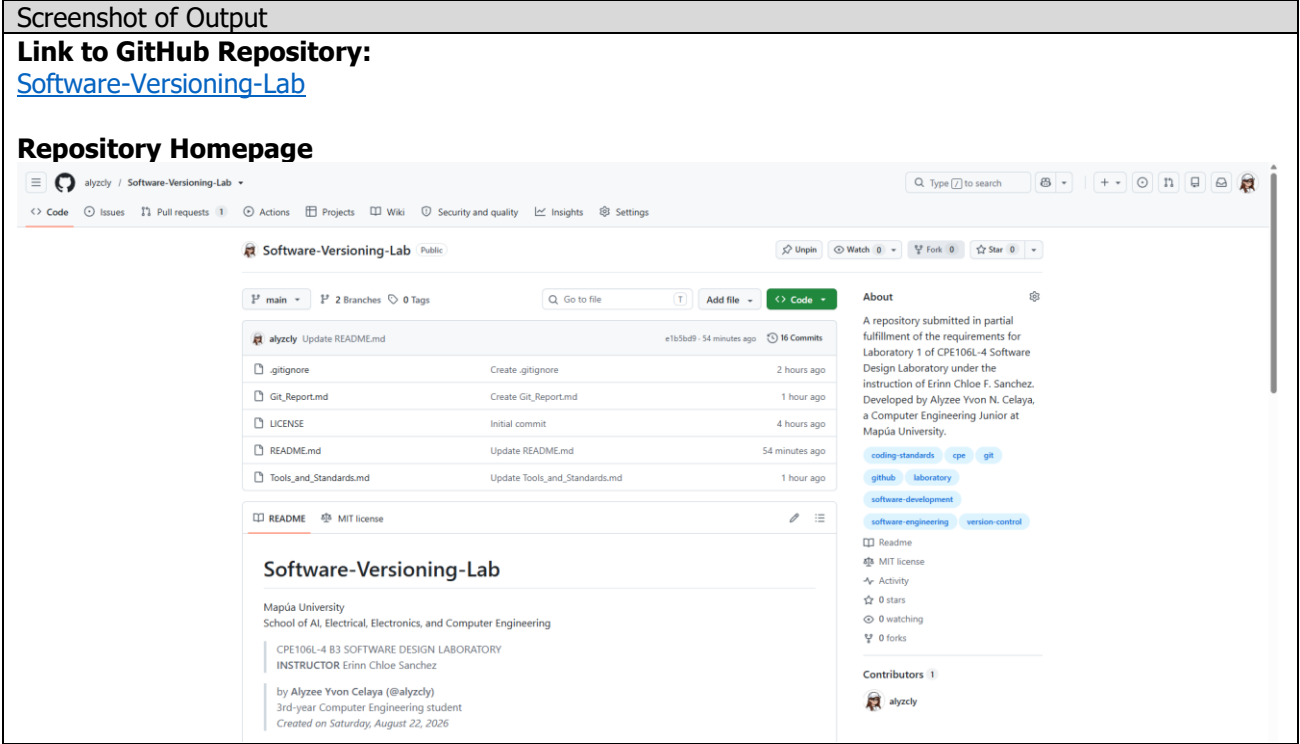
iii. .gitignore

d. Upload this laboratory file in this repository once accomplished.

e. Submit GitHub repository link

f. Screenshot of repository homepage

(Please include the following: 1. Screenshots of the output)



3. Git Version Control
- a. Perform the following Git commands and document them in a file named: (Git_Report.md)

i. Clone Repository

a) git clone <repository-url>

ii. Creating New Branch

a) git checkout -b feature-update

Software Relevant Tools, Standards, and Code Versioning using GitHub

- iii. Modify README.md
 - a) Add your name and laboratory information
- iv. Commit Changes
 - a) git add .
 - b) git commit -m "Updated README with laboratory information"
- v. Push Changes
 - a) git push origin feature-update
- vi. Create Pull Request
 - a) feature-update → main
- b. Screenshot the following:
 - i. Commands executed
 - ii. Pull Request
 - iii. Commit history

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

i. Clone Repository

```
a)
MINGW64:/c/Users/Admin

Admin@Johaness-Daryl-PC MINGW64 ~
$ git --version
git version 2.55.0.windows.5

Admin@Johaness-Daryl-PC MINGW64 ~
$ git clone https://github.com/alyzcly/Software-Versioning-Lab
Cloning into 'Software-Versioning-Lab' ...
remote: Enumerating objects: 38, done.
remote: Counting objects: 100% (38/38), done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 38 (delta 13), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (38/38), 15.85 KiB | 1014.00 KiB/s, done.
Resolving deltas: 100% (13/13), done.

Admin@Johaness-Daryl-PC MINGW64 ~
$ .....
```

ii. Creating New Branch

```
a)
Admin@Johaness-Daryl-PC MINGW64 ~/Software-Versioning-Lab (main)
$ git checkout -b feature-update
Switched to a new branch 'feature-update'

Admin@Johaness-Daryl-PC MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git branch
* feature-update
  main
```

Software Relevant Tools, Standards, and Code Versioning using GitHub

iii. Modify README.md

a)

```
① README.md X
① README.md > abc # Software-Versioning-Lab > abc ## Laboratory Tasks
1 | # Software-Versioning-Lab
2 | Mapúa University
3 | School of AI, Electrical, Electronics, and Computer Engineering
4 | > CPE106L-4 B3 SOFTWARE DESIGN LABORATORY
5 | > **INSTRUCTOR** Erinn Chloe Sanchez
6 |
7 | > by **Alyzee Yvon Celaya (@alyzcly)**
8 | > 3rd-year Computer Engineering student
9 | > _Created on Saturday, August 22, 2026_
10 |
11 | ## About
12 | This repository contains the requirements and outputs for **Laboratory 1** of **CPE106L-4 SOFTWARE DESIGN LABORATORY**
13 | The laboratory focuses on software development tools and standards, GitHub repository management, Git version control, collaboration through GitHub, and proper documentation and coding practices. The activity simulates the responsibilities of a junior software developer working in a software company where GitHub is used for source code management and software quality is maintained through coding standards and documentation.
```

iv. Commit Changes

a)

```
Admin@Johaness-Daryl-PC MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git status
On branch feature-update
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

Admin@Johaness-Daryl-PC MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git add .

Admin@Johaness-Daryl-PC MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git status
On branch feature-update
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

b)

```
Admin@Johaness-Daryl-PC MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git commit -m "Updated README with laboratory information"
[feature-update 9c7b345] Updated README with laboratory information
1 file changed, 4 insertions(+)
```

v. Push Changes

a)

```
Admin@Johaness-Daryl-PC MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git push origin feature-update
info: please complete authentication in your browser...
Enumerating objects: 41, done.
Counting objects: 100% (41/41), done.
Delta compression using up to 12 threads
Compressing objects: 100% (26/26), done.
Writing objects: 100% (41/41), 16.14 KiB | 8.07 MiB/s, done.
Total 41 (delta 15), reused 36 (delta 13), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (15/15), done.
remote:
remote: Create a pull request for 'feature-update' on GitHub by visiting:
remote:   https://github.com/alyzcly/Software-Versioning-Lab/pull/new/feature-update
remote:
To https://github.com/alyzcly/Software-Versioning-Lab
 * [new branch]      feature-update -> feature-update
```



Laboratory Report 1

Software Relevant Tools, Standards, and Code Versioning using GitHub

vi.

Create Pull Request

a)

alycly / Software-Versioning-Lab

CodeIssuesPull requestsActionsProjectsWikiSecurity and qualityInsightsSettings

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks. [Learn more about diff comparisons here.](#)

save main ← compare feature-update ✓ Able to merge. These branches can be automatically merged.

Add a title *

Updated README with laboratory information

Add a description

WritePreview

HBI<>@

This pull request updates the README.md file with the laboratory information required for Laboratory 1 of CPE106L-4 Software Design Laboratory.

Markdown is supportedPaste, drop, or click to add files

Create pull request

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

CPE106L-4 Software Design Laboratory

School of Artificial Intelligence, Electrical, Electronics, and Computer Engineering

Page | 5

Software Relevant Tools, Standards, and Code Versioning using GitHub

D. Findings, Observations, and Comments:

Guide Questions:

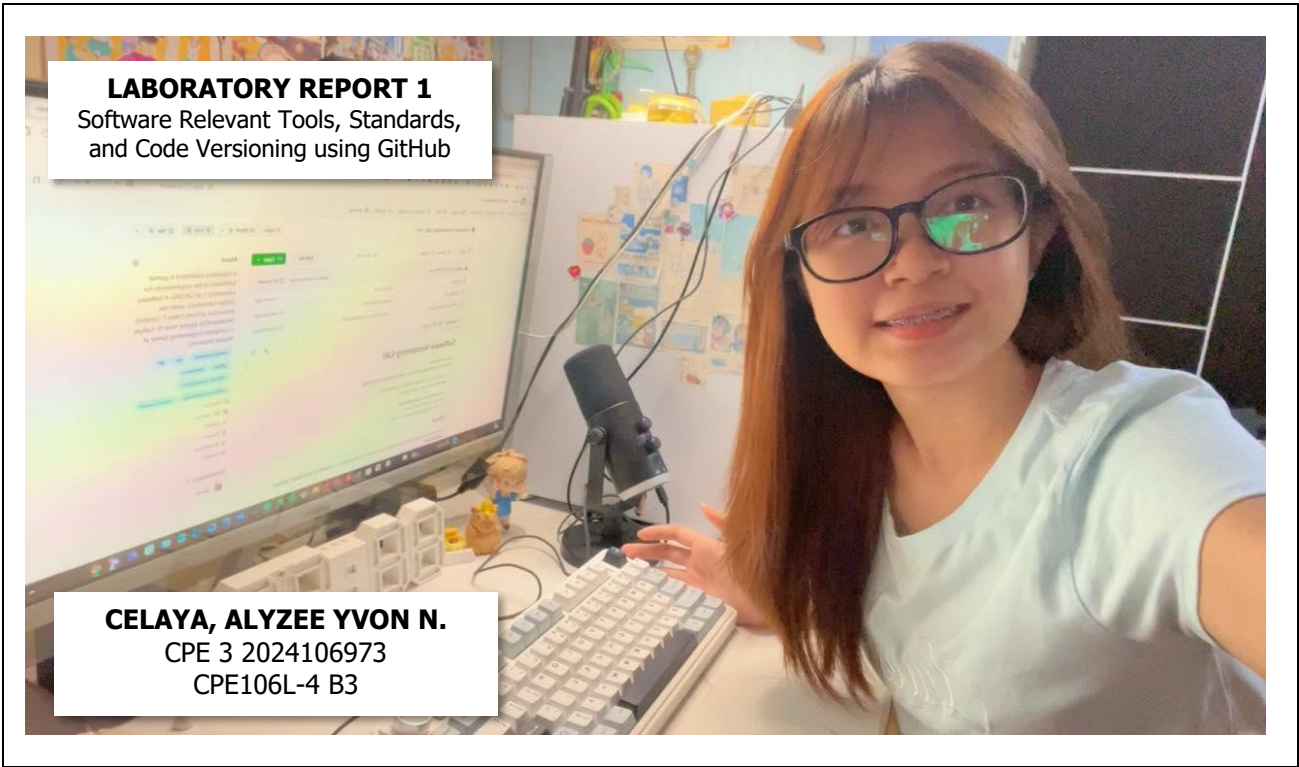
- 1. What additional steps are taken in this activity?
- 2. Are there any difficulties during the GitHub setup?
- 3. What are your findings, observations, and comments in this exercise?

Executing the required Git commands, I performed several verification and configuration steps to ensure that the workflow was properly completed. These included configuring my Git username and email, checking the active branch, verifying the repository status before and after staging changes, and confirming that the commit and push operations were successful. I also reviewed the Pull Request and commit history on GitHub to verify that the changes were correctly recorded and associated with the intended branch.

One difficulty I encountered was the Git configuration error indicating that my identity had not been set. This prevented me from committing the changes until my Git username and email were properly configured. I also initially encountered an issue with Markdown rendering because the document did not have the correct `.md` file extension. These issues were resolved through troubleshooting and verification of the repository and Git configuration.

From this exercise, I observed that Git and GitHub provide more than just a method of storing source code as Git and GitHub establishes a structured system for tracking and managing changes throughout software development. Branching allows new changes to be isolated from the main branch, while commits provide a traceable history of modifications. The Pull Request process also introduces a controlled method for reviewing and integrating changes. Overall, the activity demonstrated the importance of systematic version control, clear documentation, and verification at each stage of the development workflow. It also reinforced that troubleshooting and understanding the cause of an error are essential parts of working effectively with development tools.

PICTURE/PROOF OF DOING THE ACTIVITY





Software Relevant Tools, Standards, and Code Versioning using GitHub

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indentation, but inappropriate elements and structures and poor program documentation.	The program had corrected indentation, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indentation, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings, observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings, observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	